

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ОДЕСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ ІМЕНІ І. І. МЕЧНИКОВА
Факультет математики, фізики та інформаційних технологій Кафедра
математичного забезпечення комп'ютерних систем

КУРСОВА РОБОТА
з освітнього компоненту «Програмування»
на тему: «СТВОРЕННЯ ІЄРАРХІЇ КЛАСІВ НА ТЕМУ «ПОРТ»

Студента 1 курсу, денна форма навчання
спеціальність F7 «Комп'ютерна інженерія»

Турлакова Георгія Сергійовича

Керівник: к.т.н., доц. Шестопапов С.В.

Національна шкала: _____

Кількість балів: _____

ECTS: _____

Дата захисту: _____

Члени комісії _____

Антоненко О.С. _____

Шестопапов С.В. _____

(підпис) (прізвище та ініціали)

ОДЕСА – 2026

ЗМІСТ

ПОСТАНОВКА ЗАВДАННЯ

РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ ПРО ООП ТА ПРЕДМЕТНУ ОБЛАСТЬ

1.1 Базові концепції об'єктно-орієнтованого програмування на платформі .NET

1.2 Аналіз предметної області автоматизації портового обліку

РОЗДІЛ 2. ОПИС ЗАГАЛЬНОЇ СТРУКТУРИ ТА АЛГОРИТМУ ПРОГРАМИ

2.1 Архітектурна взаємодія та зв'язки між класами

2.2 Загальний алгоритм функціонування системи та обробки даних

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

3.1 Реалізація абстрактного базового класу **Ship**

3.2 Спеціалізовані класи-нащадки **PassengerShip** та **CargoShip**

3.3 Агрегований клас **CrewMember** та керуючий контролер **Session**

3.4 Аналіз рефакторингу коду та виправлення логічних дефектів

РОЗДІЛ 4. ДЕМОНСТРАЦІЯ ІНТЕРФЕЙСУ КОРИСТУВАЧА

ВИСНОВКИ

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

ДОДАТКИ

ПОСТАНОВКА ЗАВДАННЯ

У межах цього курсового проєкту поставлено завдання проектування та програмної реалізації об'єктно-орієнтованої автоматизованої системи обліку, контролю та оперативного управління морськими суднами в порту («Port Management System»). Застосунок має бути розроблений на базі платформи .NET із використанням мови програмування C#.

Програмні, архітектурні та функціональні вимоги до розробки:

1. Архітектурна організація: Побудова розгалуженої ієрархії класів із практичним застосуванням механізмів успадкування, інкапсуляції та поліморфізму. Головним елементом має виступати абстрактний базовий клас, який об'єднує спільні інженерні характеристики суден і координує їх глобальний статичний реєстр.
2. Функціонал предметної області: Розділення бізнес-логіки обробки даних на пасажирські та вантажні судна. Пасажирський сегмент повинен динамічно відстежувати місткість посадкових місць відносно поточної кількості людей на борту та пропонувати інтерактивний алгоритм їх розширення. Вантажний сегмент зобов'язаний здійснювати безперервний моніторинг поточної ваги вантажу з метою запобігання аварійному перевантаженню корабля.
3. Екіпаж судна: Реалізація можливості комплектації кожного окремого корабля персоналом із суворою перевіркою вікового цензу (не менше 16 років), досвіду роботи, валідності повного імені за форматом ПІБ та обов'язкової наявності капітана на борту для допуску судна до навігації.

4. Користувацький інтерфейс: Побудова текстового інтерактивного консольного меню (англійською мовою) для додавання об'єктів через парсинг CSV-рядків, модифікації параметрів у реальному часі та видалення кораблів з реєстру за унікальним індексом.
5. Надійність та стійкість: Забезпечення стійкості програми до некоректного введення даних (тексту замість чисел, від'ємних геометричних/фізичних параметрів, порожніх рядків) через генерацію та централізоване перехоплення виняткових ситуацій (**try-catch**).

РОЗДІЛ 1. ТЕОРЕТИЧНІ ВІДОМОСТІ ПРО ООП ТА ПРЕДМЕТНУ ОБЛАСТЬ

1.1 Базові концепції об'єктно-орієнтованого програмування на платформі .NET

Об'єктно-орієнтоване програмування (ООП) — це провідна парадигма розробки програмного забезпечення, де основними елементами структури є об'єкти, що взаємодіють між собою. Вона базується на чотирьох фундаментальних принципах, які повноцінно інтегровані в екосистему .NET і мову C#:

Абстракція: Виділення головних, найбільш значущих характеристик об'єкта та ігнорування другорядних деталей. У роботі абстракція реалізована через створення класу **Ship**, що містить універсальні параметри судна (наприклад, потужність силової установки, водотоннажність) без прив'язки до його конкретного комерційного чи пасажирського призначення.

Інкапсуляція: Приховування внутрішньої реалізації об'єкта та захист його полів від прямого несанкціонованого доступу чи псування ззовні. Досягається використанням модифікаторів **private** /

protected для полів класу та організацією безпечного доступу через public-властивості (**Properties** із блоками **get** і **set**) із вбудованими перевітками умов (валідацією вхідного потоку **value**).

Успадкування: Механізм, що дозволяє створювати нові класи на основі вже існуючих. Це забезпечує повторне використання коду та формування логічних ієрархій типу «є одним із». Спеціалізовані класи **CargoShip** та **PassengerShip** успадковують базову логіку та поля від абстрактного класу **Ship**, розширюючи її власними специфічними атрибутами.

Поліморфізм: Здатність об'єктів з однаковим інтерфейсом поводитися по-різному залежно від типу об'єкта під час виконання програми. У проєкті реалізовано динамічний поліморфізм через перевизначення (**override**) абстрактних методів базового класу у класах-нащадках.

1.2 Аналіз предметної області автоматизації портового обліку

Управління сучасним морським портом вимагає високого рівня автоматизації обліку діючого флоту. Кожне судно, незалежно від типу, має базові інженерні параметри: унікальний ідентифікаційний номер (індекс), назву, порт приписки, суху водотоннажність та потужність силової установки.

Проте, критерії безпеки та ефективності кардинально відрізняються для різних категорій суден. Пасажирський флот фокусується на безпеці людей, що вимагає постійного балансу між кількістю куплених квитків, чисельністю команди та фізичною наявністю обладнаних посадкових крісел. Вантажний флот оперує поняттями дедвейту та чистої вантажопідйомності: перевищення критичної норми ваги вантажу

загрожує втратою плавучості корабля. Програмний комплекс координує ці бізнес-процеси, виступаючи єдиною цифровою базою даних порту.

РОЗДІЛ 2. ОПИС ЗАГАЛЬНОЇ СТРУКТУРИ ТА АЛГОРИТМУ ПРОГРАМИ

2.1 Архітектурна взаємодія та зв'язки між класами

Проект розроблено у вигляді модульної структури, що складається з шести взаємопов'язаних класів, які взаємодіють за принципами композиції, ієрархічного успадкування та агрегації.

Базовий абстрактний клас **Ship** містить статичний список **List<Ship> ship**, який виконує роль внутрішньої оперативної бази даних порту. Об'єкти класу **CrewMember** додаються у внутрішню dynamic-колекцію **List<CrewMember> crew** кожного окремого судна, реалізуючи відношення агрегації (екіпаж може динамічно змінюватися та звільнятися, але є невід'ємною частиною життєдіяльності конкретного корабля). Класи **PassengerShip** та **CargoShip** розширюють базовий тип через пряме успадкування. Об'єкт-контролер **Session** керує потоками користувацьких вікон, а точка входу **Program** запускає головний цикл.

2.2 Загальний алгоритм функціонування системи та обробки даних

Програма працює у безкінечному циклі **while(true)**, розташованому у класі **Program**. Загальний алгоритм функціонування системи виглядає наступним чином:

1. Ініціалізація: Запуск точки входу **Main**, створення екземпляра керуючого контролера **Session**.
2. Відображення головного меню: Консоль виводить текстову панель **MAIN PORT MENU** з чотирма доступними опціями дій.
3. Обробка вибору користувача:

При виборі додавання судна (1 або 2), система запитує введення рядка даних, розділених комами (формат CSV). Програма передає цей рядок у поліморфний метод `AddShips(string character)`, де відбувається парсинг рядка, конвертація типів, перевірка регулярних виразів, генерація унікального `ShipIndex` та додавання об'єкта до загального статичного списку.

При виборі модифікації (3), викликається метод `ChooseShipYouWantToCange()` класу `Session`. Користувач вводить `ShipIndex`. Якщо судно знайдено, програма аналізує його конкретний тип за допомогою оператора `is` та скеровує користувача до відповідного спеціалізованого підменю (`ModifyPassengerShip` або `ModifyCargoShip`), де кожна опція запускає метод `ShipModification()`.

При виборі видалення (4), виконується перехід до `InterfaceforRemovingSips()`, де видалення судна з порту відбувається за його числовим індексом за допомогою команди `RemoveShipByIndex()`.

4. Валідація та перехоплення винятків: Будь-яка помилка під час виконання (наприклад, введення літер замість цифр у числових полях або порушення умов властивостей `set`) миттєво викликає генерацію винятку. Ці помилки піднімаються вгору по стеку викликів і безпечно обробляються глобальним блоком `try-catch` у `Program`, запобігаючи аварійному закриттю застосунку.

РОЗДІЛ 3. ПРОГРАМНА РЕАЛІЗАЦІЯ КОМПОНЕНТІВ СИСТЕМИ

3.1 Реалізація абстрактного базового класу `Ship`

Клас **Ship** є абстрактним фундаментом для всієї ієрархії об'єктів порту. Він інкапсулює спільні поля та визначає обов'язковий інтерфейс методів для нащадків.

Поля структури:

protected double enginePower — потужність двигуна в к.с.;

protected string name_of_the_ship — текстове ім'я судна;

protected double displacement — показник водотоннажності;

protected string name_of_the_port — назва порту приписки кораблів;

private int shipIndex — закритий системний унікальний числовий токен судна;

protected List<CrewMember> crew — динамічна колекція особового складу команди;

protected static List<Ship> ship — статичний внутрішній реєстр усіх створених об'єктів.

Властивості (Properties):

EnginePower — контролює умову **if (value < 0)**, захищаючи від встановлення від'ємної потужності двигуна.

NameOfTheShip та **NameOfThePort** — містять бізнес-перевірку **string.IsNullOrEmpty(value)** для запобігання реєстрації порожніх або безіменних рядків.

ShipIndex — public-властивість для зчитування та безпечного присвоєння токена судна.

`Crew` та `Ships` — надають public-доступ на читання до відповідних внутрішніх колекцій.

Методи класу:

`abstract Ship AddShips(string character)` — абстрактна сигнатура фабричного методу створення об'єктів із CSV-рядка.

`abstract void AddMembers(string str)` — абстрактний метод додавання та парсингу анкетних даних команди.

`abstract bool IshaCaptain()` — абстрактний логічний фільтр визначення наявності капітана на борту.

`abstract void ShipModification(int num, string modification)` — поліморфний метод покрокової зміни параметрів за номером властивості.

`RemoveShipByIndex(int index)` — універсальний пошуковий алгоритм вилучення елемента зі статичного списку за його ID через безпечний ітератор `foreach`.

3.2 Спеціалізовані класи-нащадки `PassengerShip` та `CargoShip`

Клас `PassengerShip`

Моделює поведінку та параметри пасажирських круїзних лайнерів.

Закриті поля: `number_of_passengers` (кількість пасажирів на борту), `number_of_sits` (кількість обладнаних крісел), `capacity_of_sit` (місткість одного крісла).

Властивості: `Number_of_passengers`, `Number_of_sits` (мають жорсткий фільтр контролю на від'ємні значення) та `Capacity_of_sit`.

Перевизначені методи бізнес-логіки:

`IsEnoughSits()` — повертає `true`, якщо поточна сумарна кількість людей (пасажирів + екіпаж судна з `crew.Count`) не перевищує загальну фізичну місткість пасажирського фонду кораблів: $(\text{number_of_passengers} + \text{crew.Count}) \leq \text{number_of_sits} * \text{capacity_of_sit}$.

`ChangeNumberOfSits()` — інтерактивна процедура оперативного розширення кількості місць на борту у разі виявлення дефіциту сидінь під час посадки.

`AddShips(string character)` — реалізує розбиття вхідного CSV-рядка за комами за допомогою команди `character.Split(',')`, конвертує типи та повертає новий екземпляр класу пасажирського судна.

Клас `CargoShip`

Моделює поведінку суховантажів та контейнеровозів.

Закриті поля: `loadCapacity` (максимальна вантажопідйомність), `currentLoad` (маса поточно завантаженого товару).

Властивості: `LoadCapacity` та `CurrentLoad` із вбудованими викидами винятків `throw new ArgumentException("Load can't be negative")` у разі спроби встановлення значень менше нуля.

Методи бізнес-логіки:

`IsOverloading()` — порівнює `currentLoad` та `loadCapacity`. Якщо поточна маса більша за ліміт, метод повертає `true` та сигналізує про критичне аварійне перевантаження судна.

`ShipModification(int num, string mod)` — перевизначений метод редагування, адаптований під вантажні атрибути (включаючи

автоматичний виклик `IsOverloading()` одразу після оновлення маси вантажу).

3.3 Агрегований клас `CrewMember` та керуючий контролер `Session`

Клас `CrewMember`

Описує структуру анкети кожного моряка порту.

Поля: `private string snp` (ПІБ), `protected string profession` (посада), `private int age` (вік), `private int term_of_work` (стаж).

Властивості із наскрізною валідацією даних предметної області:

`Age` — містить фільтр `if (value < 16)`. Захищає систему від порушення норм законодавства, забороняючи додавати неповнолітніх осіб до складу екіпажу судна.

`Term_of_work` — контролює стаж роботи, відсікаючи некоректні від'ємні значення.

`SNP` — містить алгоритм перевірки формату повного імені. Метод `value.Split(' ')` розбиває введений рядок по пробілах. Якщо кількість отриманих елементів менша за 3 (`str.Length < 3`) або хоча б один з них є порожнім, генерується помилка: *"Full name of the person must be defined as: Surname name patronymic"*.

`Proffession` — захищає посаду від пустих значень (`ArgumentNullException`).

Клас `Session`

Об'єкт-контролер, що координує меню управління конкретними суднами.

Методи: `ChooseShipYouWantToCange()` (перевіряє наявність кораблів у реєстрі та автоматично розгалужує інтерфейс), `ModifyPassengerShip()` та `ModifyCargoShip()` (інтерактивні консольні панелі для виклику модифікацій, додавання членів команди та перевірки наявності капітана), `InterfaceforRemovingSips()` (консольна панель для безпечного видалення суден із використанням команди `Console.Clear()`).

3.4 Аналіз рефакторингу коду та виправлення логічних дефектів

Під час фінального тестування коду було локалізовано та успішно усунено кілька критичних архітектурних та логічних помилок розробника:

1. Виправлення інкапсуляції в `PassengerShip.cs`:

Помилка: У блоці `set` властивості `Number_of_passengers` виконувалась некоректна перевірка внутрішнього поля: `if (number_of_passengers < 0)`. Через це валідація працювала на основі старих даних об'єкта, а не нового значення.

Рішення: Код виправлено на перевірку вхідного аргументу: `if (value < 0)`.

2. Виправлення втрати даних стажу роботи в `CrewMember.cs`:

Помилка: У властивості `Term_of_work` блок `set` містив лише перевірку на від'ємне значення, але інструкцію присвоєння було пропущено. Як наслідок, стаж роботи кожного моряка завжди залишався рівним 0.

Рішення: Додано рядок присвоєння: `term_of_work = value;` після блоку перевірки умов.

3. Виправлення дефекту масового видалення в `Ship.cs`:

Помилка: Метод `RemoveShipByname` здійснював підрахунок збігів імен за допомогою циклу `for (j = 0; j < ship.Count; j++)`. Проте індекс `j` після завершення циклу завжди дорівнював фінальному розміру колекції. Через це виклик видалення в класі `Session` за формулою `Count[1] - 1` призводив до видалення останнього доданого в систему корабля, замість того, чиє ім'я шукав користувач.

Рішення: Логіку видалення було повністю переведено на унікальний числовий `ShipIndex` через метод `RemoveShipByIndex`, де здійснюється безпечний перебір `foreach (Ship s in ship)` та пряме видалення об'єкта за посиланням за допомогою `ship.Remove(s)`.

4. Усунення орфографічних помилок (Typos):

Виправлено помилки в текстових винятках та інтерфейсі:
`inccorect` $\rightarrow$ `incorrect`, `proffesion` $\rightarrow$ `profession`, `peson` $\rightarrow$ `person`, `sips` $\rightarrow$ `ships`.

РОЗДІЛ 4. ДЕМОНСТРАЦІЯ ІНТЕРФЕЙСУ КОРИСТУВАЧА

Взаємодія користувача із системою «Port Management System» здійснюється через текстове інтерактивне консольне вікно. Завдяки інтеграції команд `Console.Clear()` інтерфейс очищується від застарілих даних після кожної успішної дії.

Вигляд Головного меню системи (MAIN PORT MENU):

Plaintext

```
=====
=====
```

Hi, it is your port system. Which action do you want to do?

```
=====
=====
```

```
+-----+
| MAIN PORT MENU                                |
+-----+
| 1. Add Passenger ship                        |
| 2. Add Cargo ship                          |
| 3. Change properties in your ship          |
| 4. Remove ship                            |
+-----+
```

Print number of what you want to do:

Приклад додавання пасажирського судна (Опція 1):

Користувачеві пропонується ввести параметри у форматі CSV:

Plaintext

Print characteristics of your passenger ship like: name of the ship, name of the port,
engine power, displacement, number of passengers, number of sits and capacity of sits

-> Poseidon, Odesa, 45000, 12000, 150, 200, 1

Після натискання Enter консоль очищується і виводить згенерований індекс:

Plaintext

Your ship with individual index: 1 was succesfully added to the Odesa

Вигляд підменю керування пасажирським судном (PASSENGER SHIP MANAGEMENT):

Plaintext

```
+-----+
| PASSENGER SHIP MANAGEMENT: Poseidon |
+-----+
| 1. Modify a property of your ship      |
| 2. Change number of sits of your ship  |
| 3. Add members to your ship            |
| 4. Check if the Poseidon has captain   |
| 5. Change profession of your crew member |
| 6. Exit                                |
+-----+
```

Print number of choice:

ВИСНОВКИ

Під час виконання курсового проєкту було повністю реалізовано об'єктно-орієнтовану консольну систему «Port Management System» на мові C#.

Проведена робота дозволяє зробити наступні висновки:

1. Використання абстракції та успадкування (через базовий клас **Ship** та класи-нащадки **CargoShip**, **PassengerShip**) дозволило побудувати гнучку архітектуру програми, уникнути дублювання загального коду та забезпечити легке масштабування системи в разі появи нових типів суден.
2. Принцип інкапсуляції забезпечив надійний захист даних від критичних помилок. Завдяки вбудованим перевіркам у властивостях класів **Ship** та **CrewMember**, система унеможливорює введення

некоректного віку команди (менше 16 років), від'ємної потужності двигунів, порожніх назв портів чи неправильного формату ППБ.

3. Завдяки поліморфізму реалізовано єдиний інтерфейс модифікації та динамічного парсингу об'єктів за допомогою методів `AddShips()` та `ShipModification()`.
4. Проведений рефакторинг та виправлення логічних помилок (у сеттерах пасажирів, стажу роботи та алгоритмі вилучення елементів) забезпечили високу стабільність і точність роботи застосунку. Програма успішно протидіє некоректним діям користувача завдяки централізованій обробці винятків.

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Шилдт Г. С# 4.0: полное руководство. — Пер. с англ. — М. : ООО «И.Д. Вильямс», 2011. — 1056 с.
2. Троелсен Э., Джепикс Ф. Язык программирования С# 7 и платформы .NET и .NET Core. — 8-е изд. — СПб. : Вильямс, 2018. — 1328 с.
3. Рихтер Дж. CLR via C#. Программирование на платформе Microsoft .NET Framework 4.5 на языке C#. — 4-е изд. — СПб. : Питер, 2013. — 896 с.
4. Мартин Р. Чистый код: создание, анализ и рефакторинг. — СПб. : Питер, 2019. — 464 с.